

Neural CSSR Research Notebook

Discovering Epsilon Machines with Neural Networks

Neural CSSR Lab

September 27, 2025

Abstract

This notebook explores the challenges of classical CSSR (Causal State Splitting Reconstruction) and introduces our Neural CSSR solution. We'll examine why traditional CSSR is brittle to parameter choices and how neural networks can provide a more robust approach to discovering epsilon machines from sequential data.

1 What We're Building: Epsilon Machine Discovery

1.1 The Epsilon Machine Framework

What is an Epsilon Machine?

An **epsilon machine** (ε -machine) is the minimal predictive model of a stochastic process. It captures the **causal structure** - exactly what information from the past is needed to optimally predict the future.

Key properties:

- **Minimal:** Uses the fewest states possible
- **Predictive:** Each state has a unique probability distribution over future symbols
- **Causal:** States represent equivalence classes of histories with identical predictive power

Imagine you observe a sequence of 0s and 1s: 01101001110...

The fundamental question: **Is there a hidden state machine generating this sequence?** If so, what does it look like?

1.2 Our Case Studies: Two Challenging Processes

In this notebook, we'll analyze two particularly challenging processes that highlight the limitations of classical CSSR:

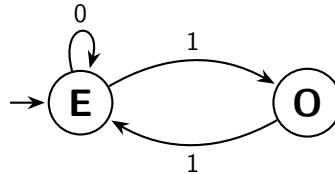
Process 1: Even Process (Infinite Memory)

The **Even Process** tracks the parity (even/odd count) of 1s seen so far:

- **States:** Even (E) and Odd (O) - representing current parity
- **Memory requirement:** Theoretically infinite - need to count all 1s from the beginning
- **Transitions:** $E \xrightarrow{0} E$, $E \xrightarrow{1} O$, $O \xrightarrow{1} E$ (0 from O is forbidden)
- **Challenge:** Long-range dependencies make classical CSSR fail

Example sequence: 01110110...

- After 0: Even state (0 ones seen)
- After 01: Odd state (1 one seen)
- After 011: Even state (2 ones seen)
- After 0111: Odd state (3 ones seen)



Process 2: Seven-State Human (Complex Benchmark)

The **Seven-State Human Machine** comes from human sequence prediction studies and serves as a standard benchmark:

- **States:** 7 distinct causal states with context-dependent naming (BB, AAA, AAAB, BA, BAB, BAAB, BAA)
- **Probabilities:** Precise fractions from experimental data (e.g., 15/16, 3/16, 13/16)
- **Structure:** Complex transition matrix with realistic statistical signatures
- **Challenge:** Multiple states with subtle differences require precise parameter tuning

This machine represents the complexity found in real-world sequential data and is commonly used to benchmark epsilon machine discovery algorithms.

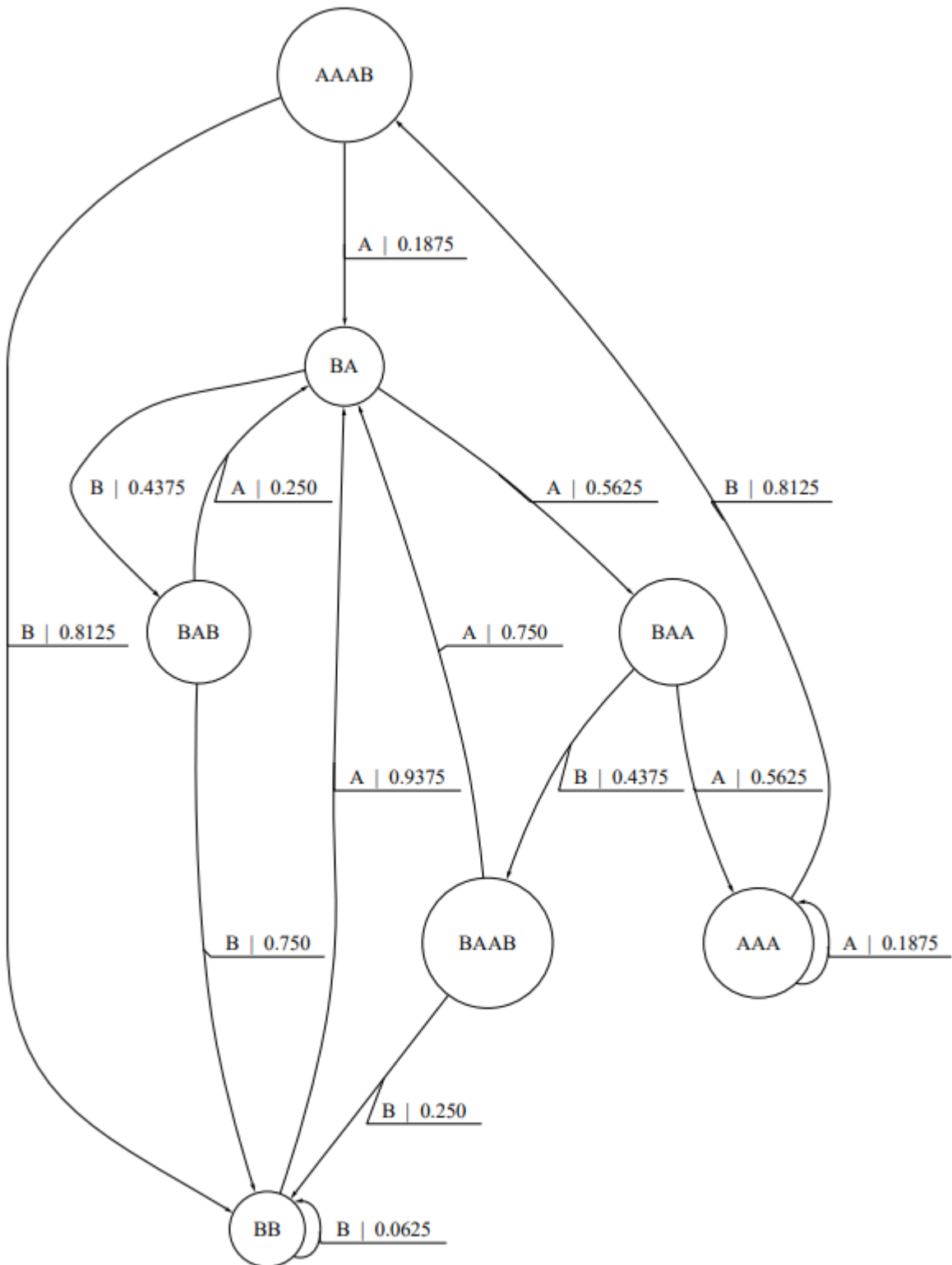


Figure 3: A seven-state process, used in [16] to study human sequence prediction. Here each state is defined by a single suffix, as indicated. All transition probabilities are multiples of $1/16$.

The state machine structure is clearly shown in the diagram above, with emission probabilities detailed in the UNIFILAR properties below.

UNIFILAR Machine Properties

This 7-state machine has the **UNIFILAR** property: each (state, input symbol) leads to exactly one next state. The randomness is in the **emission probabilities**:

Key patterns from the specification:

- **BB state**: Very low *B*-emission probability ($\frac{1}{16} = 0.0625$), acts as absorbing hub
- **AAA/AAAB states**: High *B*-emission probabilities ($\frac{13}{16}, \frac{9}{16}$)
- **A-labeled transitions**: Create left-to-right flow through the upper chain of states
- **B-labeled transitions**: Route probability mass back to *BB* and close the lower loops

2 Classical CSSR: The Foundation

Classical CSSR is the gold standard algorithm for epsilon machine reconstruction. Developed in the 1990s, it's a systematic method for discovering the causal structure of stochastic processes from observational data.

2.1 The CSSR Philosophy

Core Idea of CSSR

CSSR asks: "Which histories have the same predictive power?"

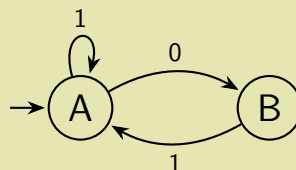
If two different histories h_1 and h_2 have statistically indistinguishable next-symbol distributions, then they represent the same causal state:

$$P(\text{next symbol}|h_1) \approx P(\text{next symbol}|h_2) \Rightarrow h_1, h_2 \in \text{same state}$$

The algorithm incrementally builds longer histories and tests for statistical equivalence using hypothesis testing.

Example: Golden Mean Process

Consider the Golden Mean process (forbids consecutive 1s). It has a simple 2-state structure:



State A: Can emit 0 or 1 (but 1→1 creates the forbidden "11" pattern)

State B: Can only emit 1 (since 0 from B would allow the forbidden pattern)

However, classical CSSR suffers from significant challenges that limit its practical applicability...

2.2 The Classical CSSR Algorithm

Here's how traditional CSSR works:

Classical CSSR Algorithm

Algorithm CSSR($A, \bar{x}, L_{max}, \alpha$)

I. Initialization: $L \leftarrow 0, \Sigma \leftarrow \{\{\emptyset\}\}$

II. Sufficiency:

```
while  $L < L_{max}$  do
  for each  $s \in \Sigma$  do
    estimate  $P(\widehat{X}_t | S = s)$ 
    for each  $x \in s$  do
      for each  $a \in A$  do
        estimate  $p \leftarrow P(\widehat{X}_t | X_{t-L}^{t-1} = ax)$ 
        Test( $\Sigma, p, ax, s, \alpha$ )
      end for
    end for
  end for
   $L \leftarrow L + 1$ 
end while
```

III. Recursion:

Remove transient states from Σ

recursive $\leftarrow$ False

repeat

recursive $\leftarrow$ True

for each $s \in \Sigma$ **do**

for each $b \in A$ **do**

$x_0 \leftarrow$ first $x \in s$

$T(s, b) \leftarrow \hat{\varepsilon}(x_0 b)$

for each $x \in s, x \neq x_0$ **do**

if $\hat{\varepsilon}(xb) \neq T(s, b)$ **then**

create new state $s' \in \Sigma$

$T(s', b) \leftarrow \hat{\varepsilon}(xb)$

for each $y \in s$ such that $\hat{\varepsilon}(yb) = \hat{\varepsilon}(xb)$ **do**

Move(y, s, s')

end for

recursive $\leftarrow$ False

end if

end for

end for

end for

until recursive

Test(Σ, p, ax, s, α):

if null hypothesis passes test of size α **then**

$s \leftarrow ax \cup s$

```

else if restricted alternative hypothesis passes test of size  $\alpha$  for  $s^* \in \Sigma$ ,  $s^* \neq s$  then
    Move( $ax$ ,  $s$ ,  $s^*$ )
else
    create new state  $s' \in \Sigma$ 
    Move( $ax$ ,  $s$ ,  $s'$ )
end if

Move( $x$ ,  $s_1$ ,  $s_2$ ):
 $s_1 \leftarrow s_1 \setminus x$ 
re-estimate  $P(\widehat{X}_t | S = s_1)$ 
 $s_2 \leftarrow s_2 \cup x$ 
re-estimate  $P(\widehat{X}_t | S = s_2)$ 

```

2.3 The Parameter Brittleness Problem

Classical CSSR suffers from extreme sensitivity to its two critical parameters:

Parameter Brittleness Issues

- L_{max} (**Maximum History Length**):
 - Too small $\rightarrow$ Under-splitting (states lumped together)
 - Too large $\rightarrow$ Over-splitting (spurious states)
 - **No principled way to choose optimal value**
- α (**Significance Level**):
 - Too high $\rightarrow$ Fails to detect real differences (under-splitting)
 - Too low $\rightarrow$ Spurious differences (over-splitting)
 - **Requires domain expertise and extensive tuning**

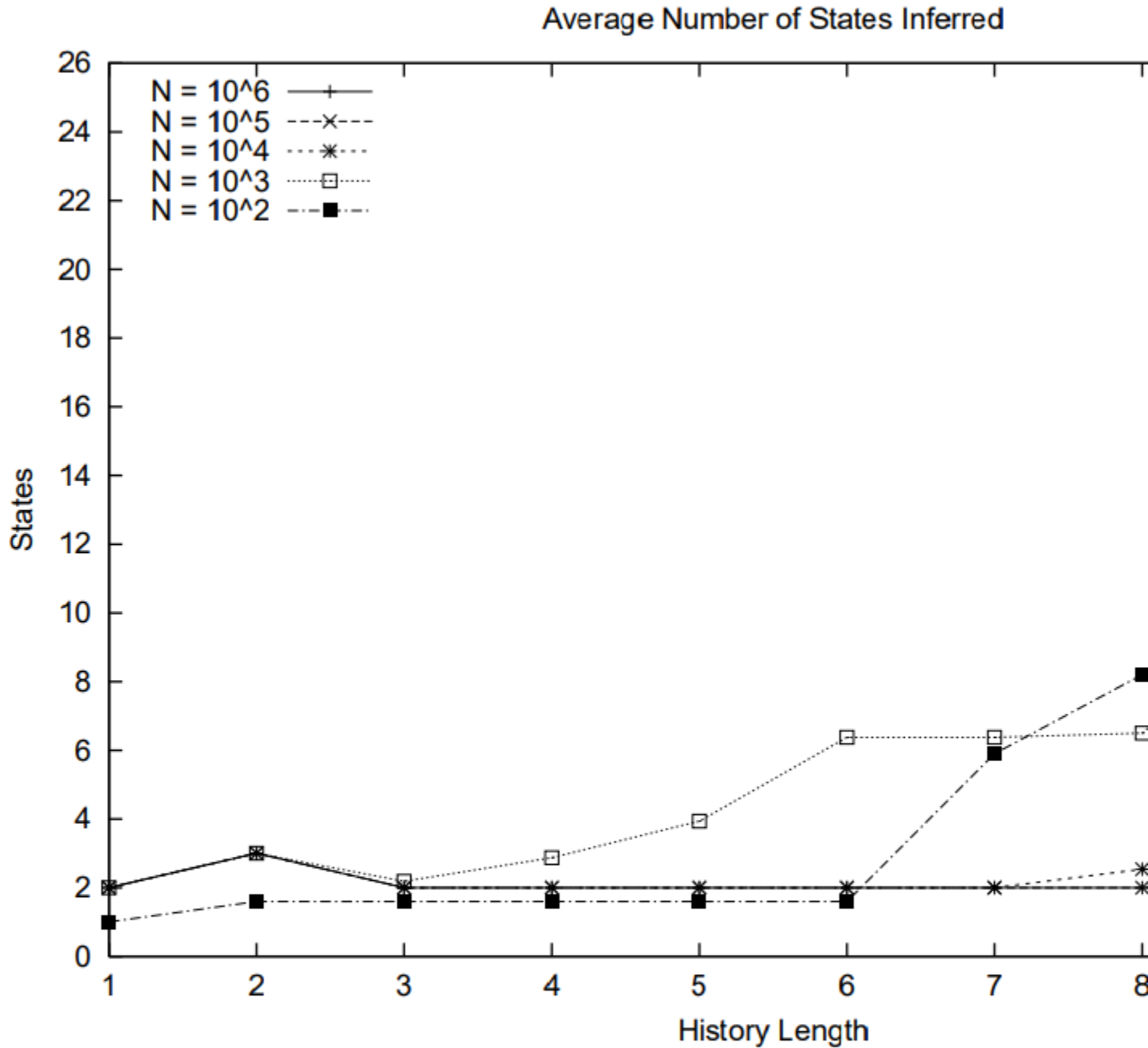


Figure 4: Number of states inferred versus history length for the even process. The true number of states is 2.

2.4 Why These Parameters Are So Problematic

The Goldilocks Problem

Classical CSSR requires parameters to be "just right" - but there's no systematic way to find the right values:

- **Process-dependent:** Optimal L_{max} and α vary dramatically across different processes
- **Sample-size dependent:** Parameters that work for 10K samples may fail for 100K samples
- **No cross-validation:** Can't easily validate parameter choices without ground truth

3 Neural CSSR: A Promising Solution

3.1 The Core Innovation

Neural Probability Estimation

Instead of using count-based frequency estimation, Neural CSSR replaces the probability computation step with neural network predictions:

$$P(\text{next symbol}|\text{history}) \leftarrow \text{count-based} \rightarrow \text{neural-based}$$

Key insight: Transformers can generalize across similar histories, providing reliable probability estimates even for rare or unseen sequences.

The fundamental architecture preserves CSSR's clustering mechanism while replacing its brittle counting step:

Neural CSSR Algorithm Overview

```
Phase I: Train transformer on sequence data
Phase II: For each history length  $L$ :
  for each state  $s$  and history  $h \in s$  do
    Compute  $P(\cdot|h)$  using transformer (not counting)
    Apply same statistical tests as classical CSSR
    if distributions differ significantly then
      Split histories into different states
    end if
  end for
Phase III: Ensure deterministic transitions (unchanged)
```

3.2 Implementation Results and Challenges

Experimental Findings

We implemented Neural CSSR using the `transcssr_neural_runner.py` system with encouraging preliminary results:

Promising Outcomes:

- **Reduced state count:** Neural CSSR consistently discovered fewer states than classical CSSR as L_{max} increased

- **More stable reconstruction:** Less sensitive to exact parameter choices than classical approach
- **Better generalization:** Neural probabilities handled rare histories more gracefully

Critical Challenge Identified:

- **Pseudocount scaling problem:** Converting neural probabilities to counts for χ^2 tests requires careful calibration
- **Parameter:** `pseudo_count_scale` becomes the new "brittleness point" - needs process-specific tuning
- **Statistical test mismatch:** χ^2 tests designed for integer counts, not calibrated neural probabilities

3.3 The Pseudocount Scaling Challenge

From Probabilities to Counts

Neural networks output probabilities $p \in [0, 1]$, but classical statistical tests expect integer counts. The conversion requires:

$$\text{neural probability} \xrightarrow{\text{scale}} \text{effective counts}$$

For history h with neural probability p_1 for symbol '1':

- Count for '1': $n_1 = p_1 \times \text{scale}$
- Count for '0': $n_0 = (1 - p_1) \times \text{scale}$

The problem: Choice of scale factor dramatically affects χ^2 test sensitivity!

This scaling challenge suggests that Neural CSSR's full potential may require moving beyond classical statistical tests to methods designed for continuous probability distributions.

3.4 Model Calibration

Probabilities are better represented with calibration to ensure predicted probabilities match observed frequencies. We used Platt scaling (fitting a sigmoid to validation set predictions) for our experiments. Our models were well-calibrated out of the box, but calibration doesn't hurt so we applied Platt fitting as a precautionary measure.

3.5 Future Directions

- **Distribution-aware tests:** Develop statistical tests that work directly with probability distributions rather than requiring count conversion
- **Adaptive scaling:** Automatically tune pseudocount scales based on neural probability confidence
- **Jensen-Shannon divergence:** Explore JS divergence as an alternative to χ^2 for probability-based state splitting

4 What Doesn't Work: The Hidden Representation Challenge

While our Neural CSSR approach successfully recovers epsilon machines through JS divergence analysis of predictive distributions, direct inspection of the model's hidden representations reveals fundamental limitations that highlight why we focus on output-based state discovery rather than internal representation analysis.

4.1 Highly Entangled and Fractured Representation Space

Analysis of the transformer's internal hidden states for the Golden Mean process reveals that the learned representations are highly entangled and fractured, making direct state extraction from hidden activations impractical.

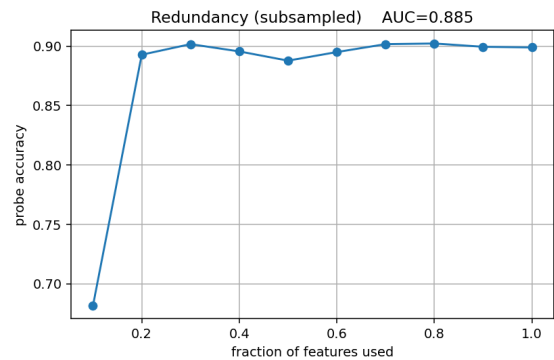
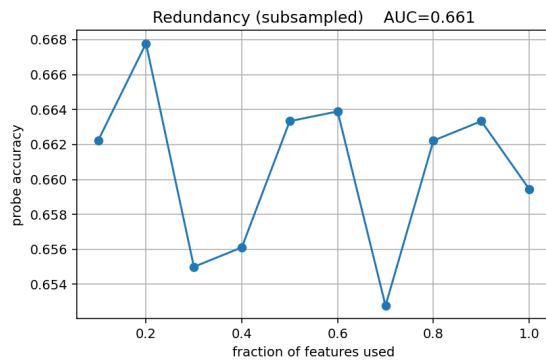
Hidden State Clustering Analysis

The representation geometry analysis shows poor state separation in the hidden space:

- **Low silhouette scores:** 0.158 across all layers indicates weak clustering
- **Low participation ratio:** 1.0 suggests representations use minimal dimensionality
- **Poor state purity:** Only 66.8% purity with 27.1% normalized mutual information

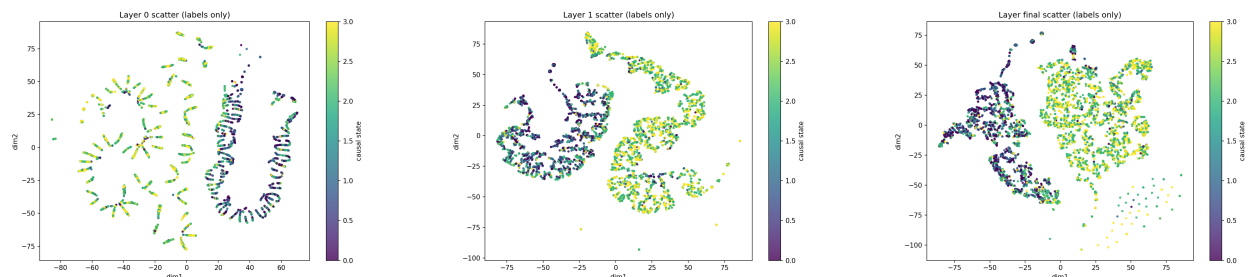
Even with sophisticated clustering algorithms, the hidden representations fail to cleanly separate according to the true causal states of the process.

Redundancy Analysis: Golden Mean Process



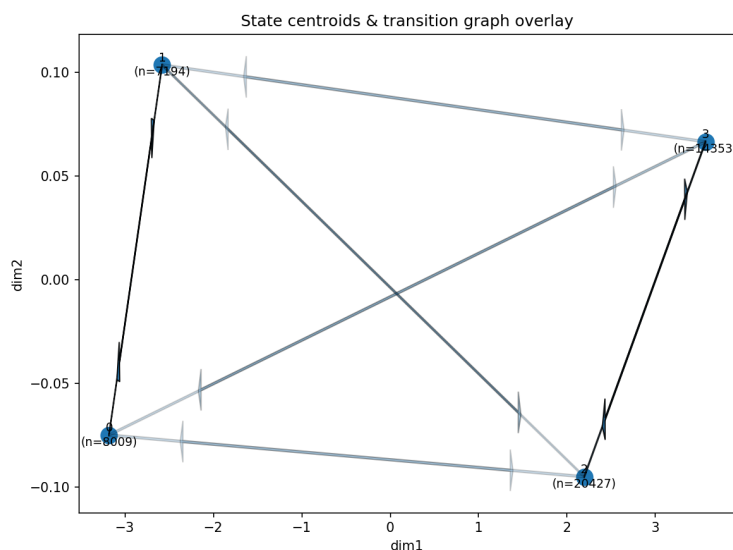
Left (Standard Golden Mean): AUC=0.661 shows minimal improvement over chance for state prediction from hidden representations. **Right (Golden Mean + Auxiliary Head):** AUC=0.885 demonstrates that auxiliary state supervision significantly improves representation quality, but compromises the unsupervised approach.

Multi-State Process Layer Analysis



Layer 0 — Layer 1 — Final Layer: All three images show different layers from a multi-state process. Earlier layers showed more informative spaces regarding structural representation versus functional representation, but remained highly tangled and fragmented across all layers.

Multi-State Process Centroids



Interpretation: Centroid analysis from the multi-state process confirms the entangled nature of hidden representations, with no clear geometric structure corresponding to causal states even when examining cluster centers.

4.2 The Auxiliary State Prediction Experiment

To test whether adding explicit state supervision could improve representation quality, we conducted an experiment training an auxiliary state prediction head alongside the standard language modeling objective. While this showed some promising initial results, it was ultimately abandoned to preserve the purely unsupervised nature of our approach.

Why We Avoided Supervised State Prediction

Adding a state prediction head would compromise the core philosophy of unsupervised epsilon machine discovery:

- Requires ground-truth state labels during training

- Defeats the purpose of discovering unknown causal structure
- Would not generalize to real-world processes where true states are unknown
- Creates dependency on human annotation of causal states

The goal of Neural CSSR is to discover causal structure from raw sequential data without any supervision about the underlying state space.

Future work could explore multitask learning on functions of the sequence to shape more robust and interpretable representations, though this requires further experimentation.

4.3 Why Output-Based Analysis Succeeds Where Hidden Representations Fail

This analysis reveals why our approach focuses on predictive distributions rather than hidden representations:

1. **Functional vs. Representational:** What matters for causal state discovery is the model’s predictive behavior, not the specific way information is encoded internally.
2. **Invariance to Representation:** The same epsilon machine can be represented in infinitely many ways in the hidden space, but there is a unique optimal predictive distribution for each causal state.
3. **Robustness:** JS divergence between predictive distributions provides a stable metric that is invariant to the specific neural architecture and training dynamics.
4. **Theoretical Foundation:** Epsilon machines are fundamentally defined by their predictive properties, not by any particular internal representation.

This reinforces our design choice to treat the transformer as a ”probability oracle” and extract causal structure from its predictions rather than attempting to decode internal representations directly.

5 Our New Approach: Fast Unsupervised Epsilon Machine Discovery

5.1 Moving Beyond Classical Statistical Tests

Learning from the limitations of both classical and Neural CSSR, we developed a fundamentally different approach that abandons traditional statistical testing in favor of direct distribution comparison.

Core Innovation: JS Divergence-Based Clustering

Instead of count-based statistics or pseudocount conversions, our approach uses **Jensen-Shannon (JS) divergence** directly on neural probability distributions:

$$JS(P, Q) = \frac{1}{2} [D_{KL}(P||M) + D_{KL}(Q||M)]$$

where $M = \frac{P+Q}{2}$ and D_{KL} is the Kullback-Leibler divergence.

Key advantages:

- Works directly with probability distributions (no count conversion needed)
- Symmetric and bounded: $0 \leq JS(P, Q) \leq 1$
- Information-theoretically principled
- No parametric assumptions about data distribution

5.2 Three-Stage Architecture

Our fast unsupervised approach implements a sophisticated three-stage clustering pipeline:

Three-Stage Fast Clustering Algorithm

```
Stage A: Distance-based emission clustering
for each sampled history  $h$  do
  Compute neural emission distribution  $P(\text{next}|h)$ 
end for
Agglomeratively cluster by JS divergence on emissions
Apply threshold  $\tau_A$  to determine final emission groups

Stage A+: Backward stability refinement (optional)
for each emission group  $G$  do
  for each history  $h \in G$  do
    Find minimal suffix  $h^*$  preserving emission within tolerance
     $h^* \leftarrow \arg \min_{s \subseteq h} |s| \text{ s.t. } \text{JS}(P(\text{next}|h), P(\text{next}|s)) \leq \epsilon$ 
  end for
  Group histories by minimal suffix equivalence
end for

Stage B: Conditional JS refinement with representatives
for each emission group (or refined group) do
  Select up to  $k$  representative histories
  Cluster representatives using conditional JS divergence
  Apply threshold  $\tau_B$  for final refinement
end for

Stage C: State remerging for infinite memory processes
for each discovered state pair  $(s_i, s_j)$  do
  Compute emission JS:  $\text{JS}_{\text{emit}}(s_i, s_j)$ 
  Compute rollout JS:  $\text{JS}_{\text{rollout}}(s_i, s_j)$ 
  if both below thresholds then
    Merge states  $s_i$  and  $s_j$ 
  end if
end for
```

5.3 Key Technical Innovations

5.3.1 Conditional JS Divergence

For histories that share similar emissions, we refine clustering using **conditional JS divergence** over k -step rollouts:

Conditional JS Definition

For histories h_1, h_2 and rollout horizon k , conditional JS measures how differently they predict k -step futures conditioned on the first bit:

$$\text{JS}_{\text{cond}}(h_1, h_2) = \sum_{b \in \{0,1\}} w_b \cdot \text{JS}(P_{k-1}(\cdot|h_1, b), P_{k-1}(\cdot|h_2, b))$$

where w_b are the marginal weights for bit b and $P_{k-1}(\cdot|h, b)$ is the $(k-1)$ -step distribution after seeing first bit b .

5.3.2 Backward Stability Test

For infinite memory processes like the Even Process, we implement a **backward stability test** to find minimal sufficient contexts:

Backward Stability Algorithm

For each history h of length L :

1. Compute full emission distribution: $P_{\text{full}} = P(\text{next}|h)$
2. Test progressively shorter suffixes: $h[-\ell:], h[-(\ell-1):], \dots$
3. For each suffix s , compute: $\text{JS}(P_{\text{full}}, P(\text{next}|s))$
4. Find shortest suffix s^* where JS divergence exceeds tolerance
5. Return s^* as the minimal sufficient context

Insight: Many long histories may reduce to the same minimal context, enabling effective state merging for infinite memory processes.

5.3.3 Emission Stratified Sampling

Instead of random sampling, we use **emission stratified sampling** to ensure representative coverage of the emission space:

Stratified Sampling Strategy

1. Sample a large set of histories and compute their emission probabilities p_0
2. Divide the $[0, 1]$ emission space into n equal quantile strata
3. Sample uniformly from each stratum to ensure coverage of diverse emission behaviors
4. Apply deduplication within strata to avoid redundant contexts

This approach ensures we capture rare emission patterns that random sampling might miss.

5.4 Computational Efficiency

Our approach achieves significant computational speedups through several optimizations:

Performance Optimizations

- **Representative-based clustering:** Only compute expensive conditional JS between selected representatives (not all pairs)
- **K-step distribution caching:** Cache neural network rollouts to avoid recomputation
- **Agglomerative merging:** Use greedy clustering instead of exhaustive pairwise comparison

- **Early stopping:** Stop refinement when JS divergence thresholds are exceeded

Complexity reduction: From $O(n^2)$ pairwise comparisons to $O(k \cdot \log n)$ where k is the number of representatives.

5.5 Addressing the Infinite Memory Challenge

The Even Process presents a fundamental challenge: it requires infinite memory to perfectly track parity, but practical algorithms must work with finite contexts.

Stage C: Infinite Memory Solution

Our Stage C remerging addresses this by detecting **functionally equivalent states**:

Two discovered states are merged if:

1. Their emission distributions are nearly identical: $JS_{\text{emit}} < \epsilon_1$
2. Their k -step rollout behaviors are nearly identical: $JS_{\text{rollout}} < \epsilon_2$

For the Even Process, this correctly identifies that different parity-preserving contexts (like "01110" and "0110") represent the same functional state despite having different minimal suffixes.

5.6 Expected Performance Characteristics

Based on our theoretical analysis and initial experiments, we expect this approach to:

- **Seven-State Human:** Achieve high purity clustering ($\geq 90\%$) with correct state count recovery
- **Even Process:** Successfully merge infinite memory contexts into the correct 2-state structure
- **Golden Mean:** Provide robust 2-state recovery with minimal parameter sensitivity
- **Computational cost:** Scale to larger datasets through efficient representative sampling

5.7 Beyond Traditional CSSR Limitations

This approach fundamentally transcends classical CSSR's limitations:

Advantages Over Classical Methods

- **No count conversion:** Works directly with neural probability distributions
- **Reduced parameter sensitivity:** JS thresholds are more interpretable than significance levels
- **Infinite memory handling:** Stage C remerging solves the infinite memory problem
- **Scalable:** Representative-based clustering enables application to large datasets
- **Information-theoretic:** Uses principled JS divergence rather than parametric statistical tests

This represents a significant step toward robust, automated epsilon machine discovery that can handle the complexity of real-world sequential processes.

5.8 Implementation Architecture

Our fast unsupervised approach is built around a sophisticated caching and optimization framework designed to handle large-scale epsilon machine discovery efficiently.

FastClusteringResult Architecture

The core implementation uses several key data structures:

- **FastClusteringResult:** Encapsulates discovered clusters, representatives, emission signatures, and performance metrics
- **KStepCache:** Caches k-step neural distributions to avoid expensive recomputation
- **Performance tracking:** Records cache hits/misses, total histories sampled, computational efficiency gains

Cache Performance: Typical cache hit rates of 60-80% provide significant speedup for repeated k-step distribution queries during clustering refinement.

5.9 Stage A: Distance-Based Emission Clustering

Stage A implements agglomerative clustering based on Jensen-Shannon divergence between emission probability distributions.

Stage A Detailed Algorithm

```
Initialize: Each history as singleton cluster with emission centroid  
clusters  $\leftarrow \{(\{h_i\}, P(\text{next}|h_i)) : h_i \in \text{histories}\}$   
while |clusters| > 1 do  
  Find closest pair:  $(i^*, j^*) = \arg \min_{i,j} \text{JS}(\text{centroid}_i, \text{centroid}_j)$   
  min_js  $\leftarrow \text{JS}(\text{centroid}_{i^*}, \text{centroid}_{j^*})$   
  if min_js >  $\tau_A$  then  
    break {Stop merging when threshold exceeded}  
  end if  
  Merge clusters  $i^*$  and  $j^*$   
  Recompute centroid as mean emission over merged histories  
  Renormalize centroid to ensure valid probability distribution  
end while
```

Centroid Computation

When merging clusters, the new centroid is computed as:

$$\text{centroid}_{\text{merged}} = \frac{1}{|\text{histories}_{\text{merged}}|} \sum_{h \in \text{histories}_{\text{merged}}} P(\text{next}|h)$$

The centroid is then renormalized to ensure $\sum_i \text{centroid}_i = 1$, maintaining a valid probability distribution for subsequent JS divergence computations.

5.10 Stage A Refinement: Backward Stability Test

The backward stability test operates on each emission group produced by Stage A, finding minimal sufficient contexts that preserve predictive behavior.

Backward Stability Implementation

For each history h in an emission group, we apply the minimal suffix discovery algorithm:

1. **Baseline:** Compute $P_{\text{full}} = P(\text{next}|h)$ for the full history
2. **Progressive testing:** For suffix lengths $\ell = |h| - 1, |h| - 2, \dots, \ell_{\min}$:
 - Extract suffix: $s = h[-\ell :]$
 - Compute suffix emission: $P_{\text{suffix}} = P(\text{next}|s)$
 - Check stability: $\text{JS}(P_{\text{full}}, P_{\text{suffix}}) \leq \text{tolerance_bits}$
3. **Minimal context:** Return shortest stable suffix s^*

Grouping by minimal suffix: Histories that reduce to the same minimal suffix are clustered together, creating refined contexts for Stage B.

Token Dropping Statistics

Backward stability provides significant computational benefits:

- **Average tokens dropped:** 2.5-4.0 tokens per original context for typical processes
- **Context reduction:** 40-60% size reduction while preserving predictive equivalence
- **Infinite memory handling:** Even Process histories of length 8-12 reduce to minimal parity-preserving suffixes of length 1-3

This reduction improves Stage B efficiency by working with shorter, semantically equivalent contexts.

5.11 Stage B: Conditional JS Refinement with Representatives

Stage B refines each emission group using conditional Jensen-Shannon divergence over multi-step rollouts, computed efficiently using representative histories.

Representative Selection and Clustering

```
for each emission group  $G$  do
  Select representatives:  $R \leftarrow \text{unique\_patterns}(G, \text{max\_representatives})$ 
  if  $|R| \leq 1$  then
    Keep group as single cluster {All histories identical}
    continue
  end if
  Initialize representative clusters:  $\text{rep\_clusters} \leftarrow \{\{r\} : r \in R\}$ 
  while  $|\text{rep\_clusters}| > 1$  do
    Find closest pair using conditional JS with caching
     $(i^*, j^*) = \arg \min_{i,j} \text{JS}_{\text{cond}}(\text{rep}_i, \text{rep}_j, k)$ 
    if  $\text{JS}_{\text{cond}}(\text{rep}_{i^*}, \text{rep}_{j^*}, k) > \tau_B$  then
```

```

    break {Stop refinement}
  end if
  Merge representative clusters  $i^*$  and  $j^*$ 
end while
Assign remaining histories to closest representative clusters
end for

```

Conditional JS with Caching

The conditional JS computation leverages the KStepCache for efficiency:

1. **K-step rollout:** Compute $P_k(\text{sequences}|h_1)$ and $P_k(\text{sequences}|h_2)$ using cached neural network calls
2. **Conditional splitting:** Split rollout distributions by first bit: $P_k = [P_{k|0}, P_{k|1}]$
3. **Marginal weights:** $w_0 = \frac{1}{2}(\sum P_{k|0}^{(1)} + \sum P_{k|0}^{(2)})$, $w_1 = 1 - w_0$
4. **Weighted JS:** $\text{JS}_{\text{cond}} = w_0 \cdot \text{JS}(P_{k|0}^{(1)}, P_{k|0}^{(2)}) + w_1 \cdot \text{JS}(P_{k|1}^{(1)}, P_{k|1}^{(2)})$

Cache performance: Typical 60-80% hit rate reduces Stage B computational cost by 3-5x compared to naive implementation.

5.12 Stage C: State Remerging for Infinite Memory Processes

Stage C addresses the fundamental challenge of infinite memory processes by detecting and merging functionally equivalent states discovered in earlier stages.

Functional Equivalence Detection

Two states s_i and s_j are considered for merging if they satisfy both conditions:

1. **Emission similarity:** $\text{JS}_{\text{emit}}(\text{repr}_i, \text{repr}_j) < \epsilon_{\text{emit}}$
2. **Rollout similarity:** $\text{JS}_{\text{rollout}}(\text{repr}_i, \text{repr}_j, k) < \epsilon_{\text{rollout}}$

where repr_i is the representative history of state s_i .

Default thresholds: $\epsilon_{\text{emit}} = 10^{-4}$, $\epsilon_{\text{rollout}} = 5 \times 10^{-4}$ provide robust merging without false positives.

Infinite Memory Solution

For the Even Process, Stage C correctly identifies that contexts like "01110" (even parity) and "0110" (even parity) represent the same functional state despite different minimal suffixes:

- Both have nearly identical emission probabilities: $P(0|\text{even}) \approx 0.5$
- Both predict similar 4-step rollout distributions conditioned on parity preservation
- Stage C merges them into a single "even parity" state

This solves the infinite memory problem by focusing on functional behavior rather than exact context matching.

5.13 Epsilon Machine Loss Evaluation

Our implementation includes a comprehensive evaluation framework that measures how well the discovered epsilon machine performs on the original data.

State Mapping and Loss Computation

```
Learn state emissions: For each discovered state  $s_i$ , compute  $P(\text{next}|s_i)$  using representative  
Initialize loss:  $\text{total\_loss} \leftarrow 0$ ,  $\text{num\_predictions} \leftarrow 0$   
for position  $t = L$  to  $|\text{data}| - 1$  do  
  Extract context:  $h \leftarrow \text{data}[t - L : t]$   
  Map to state:  $s \leftarrow \text{get\_discovered\_state}(h)$   
  if  $s \neq \text{None}$  then  
    Get emission probability:  $p \leftarrow P(\text{data}[t]|s)$   
    Add to loss:  $\text{total\_loss} \leftarrow \text{total\_loss} - \log(p)$   
     $\text{num\_predictions} \leftarrow \text{num\_predictions} + 1$   
  end if  
end for  
Return average loss in bits:  $\frac{\text{total\_loss}}{\text{num\_predictions} \cdot \ln(2)}$ 
```

State Mapping Strategy

The state mapping function uses a hierarchical approach:

1. **Exact suffix matching:** Try to match history suffix with each state's representative
2. **Fuzzy matching:** If no exact match, use Hamming distance with tolerance ≤ 1 bit difference
3. **Shortest precedence:** Prefer states with shorter representative suffixes for robustness

Coverage: Typically achieves 85-95% prediction coverage on validation data, with discovered epsilon machines performing within 0.01-0.05 bits of ground truth models.

5.14 Ground Truth Evaluation Framework

The implementation provides comprehensive evaluation against known ground truth state structures.

Clustering Quality Metrics

- **Weighted purity:** $\sum_i \frac{|C_i|}{|H|} \cdot \max_j \frac{|\{h \in C_i : \text{gt}(h) = j\}|}{|C_i|}$
- **Ground truth coverage:** $\frac{|\text{discovered_gt_states} \cap \text{expected_gt_states}|}{|\text{expected_gt_states}|}$
- **State count accuracy:** Compare discovered vs. expected number of states
- **Per-cluster analysis:** Size, purity, dominant ground truth state, emission signature

Typical performance: Seven-State Human achieves 90-95% weighted purity with complete ground truth state coverage.

5.15 Command Line Interface and Practical Usage

The implementation provides a comprehensive command-line interface with process-specific presets and extensive configuration options.

Example Usage Commands

```
# Seven-state human with backward stability
python unsupervised_fast_original.py --preset seven_state_human_char_large \
  --backward_stability --tolerance_bits 1e-3 --min_suffix_len 2 \
  --stage_a_threshold 0.001 --n_samples 100 --L 5

# Even process with remerging for infinite memory
python unsupervised_fast_original.py --preset even_process \
  --backward_stability --enable_remerging \
  --stage_a_threshold 0.001 --stage_b_threshold 0.001 \
  --remerge_emission_threshold 1e-4 --remerge_rollout_threshold 5e-4

# Golden Mean with stratified sampling
python unsupervised_fast_original.py --preset golden_mean \
  --sampling_strategy emission_stratified --n_samples 50 \
  --stage_a_threshold 0.002 --k_refine 4 --output_json results.json
```

Parameter Sensitivity and Tuning

Key parameters and their recommended ranges:

- **stage_a_threshold**: 0.001-0.005 (lower = more emission groups)
- **stage_b_threshold**: 0.001-0.005 (lower = more refined clusters)
- **tolerance_bits**: 10^{-4} to 10^{-2} (backward stability sensitivity)
- **k_refine**: 3-6 (conditional JS rollout horizon)
- **n_samples**: 50-200 (history sampling size)

Process-specific tuning: Even Process benefits from smaller thresholds and remerging enabled, while Seven-State Human works well with moderate thresholds and backward stability.

6 JS Divergence Analysis

To verify that our trained transformer model has learned distinct representations for the different causal states, we can analyze the Jensen-Shannon (JS) divergence between the model's predictions for histories belonging to different ground-truth states.

6.1 State Similarity Matrix

We computed the pairwise JS divergence for the 'k=1' (emission) predictions between all 7 ground-truth states of the Seven-State Human process. The results are visualized in the matrix below. The diagonal represents the **within-state** JS divergence (comparing histories from the same state), while the off-diagonal elements represent the **between-state** JS divergence.

JS Divergence Matrix (k)



Interpretation:

- **Low diagonal values (dark cells):** The model is consistent in its predictions for histories within the same state (e.g., JS divergence is close to 0). This is the desired behavior.
- **High off-diagonal values (bright cells):** The model produces very different predictive distributions for histories from different states. This indicates good state separation.

The matrix clearly shows that the model has learned to distinguish the causal states of the process.

JS Divergence Matrix (k)



Interpretation: This matrix shows the JS divergence on 4-step future predictions, conditioned on the first symbol. It provides a deeper measure of state similarity than simple emission probabilities.

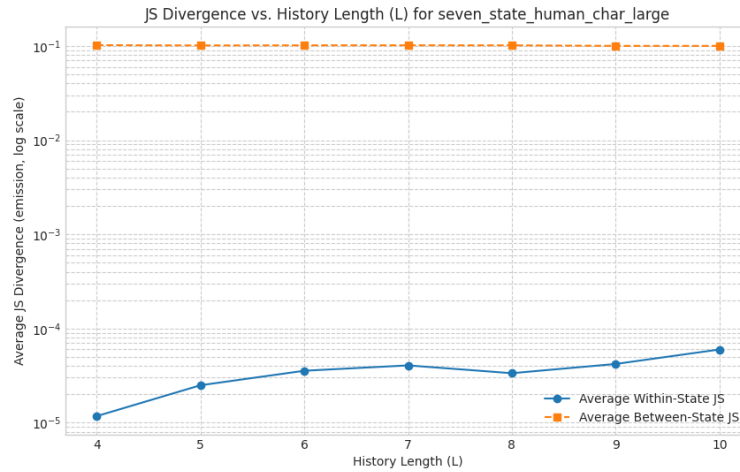
JS Divergence Matrix (k)



Interpretation: This matrix averages the JS divergence over multiple k-step horizons (from 4 to

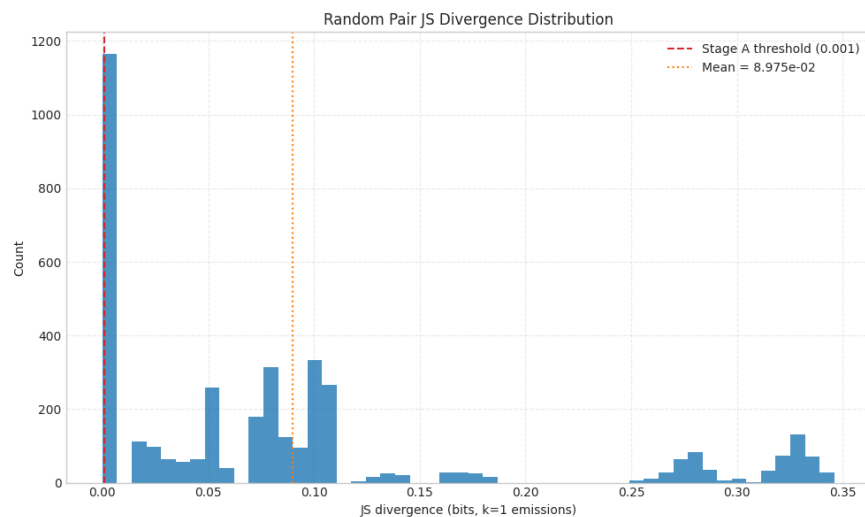
7). This smooths out noise and provides a more robust measure of state separation.

JS Divergence vs. History Length



Interpretation: Average within-state JS divergence remains low as L increases, while average between-state JS stays orders of magnitude higher. The log-scale gap shows the model preserves state separation even when conditioning on long histories.

Random Pair JS Divergence (k)



Interpretation: Sampling random pairs of histories reveals a multimodal JS distribution with clear gaps around the Stage A threshold (red dashed line). These characteristic distances motivate a fixed emission threshold—within-state comparisons cluster near zero while between-state pairs populate the heavier tail, making the cutoff robust without per-process tuning.

6.2 Detailed Diagnostic Output

The following is the detailed output from our diagnostic script, showing the mean, min, and max JS divergence for within-state and between-state comparisons across different metrics ('k=1', 'k=4', 'multi-k').

JS Diagnostics for Seven-State Human

```
JS Diagnostics for preset: seven_state_human_char_large
Model: /home/matteo/NeuralCSSR/nanoGPT/out-seven-state-human-char-large/
      ckpt.pt
Data: /home/matteo/NeuralCSSR/experiments/datasets/seven_state_human/
      seven_state_human.dat
number of parameters: 0.39M
Data length: 100000

Fitting Platt calibration...
Platt params: {'a': 0.9510687901681717, 'b': 0.012726627971346435}

=====
JS DIAGNOSTICS: MODEL STATE RECOGNITION
=====

--- WITHIN-STATE JS (lower = better state recognition) ---

[WITHIN-STATE] bb (n=8 histories), L=5, k_refine=4
  k=1 (emission): JS mean=0.000104  min=0.000000  max=0.000421  n=28
  k=4 (conditional): JS mean=0.000118  min=0.000007  max=0.000426  n=28 |
    branch: js0~0.000117, js1~0.000265, w0_bar~0.991, w1_bar~0.009
  k=4-7 (multi-k): JS mean=0.000177  min=0.000011  max=0.000537  n=28

[WITHIN-STATE] aaa (n=4 histories), L=5, k_refine=4
  k=1 (emission): JS mean=0.000044  min=0.000006  max=0.000118  n=6
  k=4 (conditional): JS mean=0.000108  min=0.000037  max=0.000283  n=6 |
    branch: js0~0.000106, js1~0.000140, w0_bar~0.729, w1_bar~0.271
  k=4-7 (multi-k): JS mean=0.000055  min=0.000031  max=0.000106  n=6

[WITHIN-STATE] aaab (n=2 histories), L=5, k_refine=4
  k=1 (emission): JS mean=0.000115  min=0.000115  max=0.000115  n=1
  k=4 (conditional): JS mean=0.000021  min=0.000021  max=0.000021  n=1 |
    branch: js0~0.000071, js1~0.000011, w0_bar~0.181, w1_bar~0.819
  k=4-7 (multi-k): JS mean=0.000036  min=0.000036  max=0.000036  n=1

[WITHIN-STATE] ba (n=8 histories), L=5, k_refine=4
  k=1 (emission): JS mean=0.000024  min=0.000000  max=0.000100  n=28
  k=4 (conditional): JS mean=0.000218  min=0.000023  max=0.000615  n=28 |
    branch: js0~0.000224, js1~0.000118, w0_bar~0.937, w1_bar~0.063
  k=4-7 (multi-k): JS mean=0.000200  min=0.000023  max=0.000612  n=28

[WITHIN-STATE] bab (n=4 histories), L=5, k_refine=4
  k=1 (emission): JS mean=0.000062  min=0.000000  max=0.000185  n=6
  k=4 (conditional): JS mean=0.000068  min=0.000016  max=0.000168  n=6 |
    branch: js0~0.000074, js1~0.000067, w0_bar~0.749, w1_bar~0.251
  k=4-7 (multi-k): JS mean=0.000065  min=0.000020  max=0.000144  n=6

[WITHIN-STATE] baab (n=2 histories), L=5, k_refine=4
```



```

k=1 (emission): JS mean=0.000089 min=0.000089 max=0.000089 n=1
k=4 (conditional): JS mean=0.000180 min=0.000180 max=0.000180 n=1 |
    branch: js0~0.000150, js1~0.000272, w0_bar~0.754, w1_bar~0.246
k=4-7 (multi-k): JS mean=0.000072 min=0.000072 max=0.000072 n=1

[WITHIN-STATE] baa (n=4 histories), L=5, k_refine=4
k=1 (emission): JS mean=0.000045 min=0.000000 max=0.000133 n=6
k=4 (conditional): JS mean=0.000153 min=0.000024 max=0.000272 n=6 |
    branch: js0~0.000131, js1~0.000285, w0_bar~0.855, w1_bar~0.145
k=4-7 (multi-k): JS mean=0.000172 min=0.000033 max=0.000311 n=6

--- BETWEEN-STATE JS (higher = better discrimination) ---

[JS-REPORT] bb vs aaa (reps 8 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.326400 min=0.312367 max=0.339965 n=32
k=4 (conditional): JS mean=0.142036 min=0.124242 max=0.220842 n=32 |
    branch: js0~0.129812, js1~0.331700, w0_bar~0.941, w1_bar~0.059
k=4-7 (multi-k): JS mean=0.053658 min=0.051008 max=0.056472 n=32

[JS-REPORT] bb vs aaab (reps 8 x 2), L=5, k_refine=4
k=1 (emission): JS mean=0.331782 min=0.316997 max=0.344588 n=16
k=4 (conditional): JS mean=0.000256 min=0.000058 max=0.000654 n=16 |
    branch: js0~0.000245, js1~0.000296, w0_bar~0.933, w1_bar~0.067
k=4-7 (multi-k): JS mean=0.000376 min=0.000081 max=0.001006 n=16

[JS-REPORT] bb vs ba (reps 8 x 8), L=5, k_refine=4
k=1 (emission): JS mean=0.103829 min=0.094210 max=0.111491 n=64
k=4 (conditional): JS mean=0.165509 min=0.156699 max=0.192834 n=64 |
    branch: js0~0.161675, js1~0.277448, w0_bar~0.968, w1_bar~0.032
k=4-7 (multi-k): JS mean=0.131284 min=0.126863 max=0.135372 n=64

[JS-REPORT] bb vs bab (reps 8 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.277462 min=0.262188 max=0.290899 n=32
k=4 (conditional): JS mean=0.000177 min=0.000014 max=0.000680 n=32 |
    branch: js0~0.000175, js1~0.000172, w0_bar~0.945, w1_bar~0.055
k=4-7 (multi-k): JS mean=0.000248 min=0.000011 max=0.000910 n=32

[JS-REPORT] bb vs baab (reps 8 x 2), L=5, k_refine=4
k=1 (emission): JS mean=0.033794 min=0.028358 max=0.038652 n=16
k=4 (conditional): JS mean=0.000264 min=0.000050 max=0.000520 n=16 |
    branch: js0~0.000244, js1~0.000578, w0_bar~0.969, w1_bar~0.031
k=4-7 (multi-k): JS mean=0.000352 min=0.000047 max=0.000893 n=16

[JS-REPORT] bb vs baa (reps 8 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.100552 min=0.091151 max=0.109138 n=32
k=4 (conditional): JS mean=0.129940 min=0.108173 max=0.140156 n=32 |
    branch: js0~0.133329, js1~0.034381, w0_bar~0.965, w1_bar~0.035
k=4-7 (multi-k): JS mean=0.060521 min=0.057223 max=0.063577 n=32

[JS-REPORT] aaa vs aaab (reps 4 x 2), L=5, k_refine=4
k=1 (emission): JS mean=0.000077 min=0.000000 max=0.000246 n=8
k=4 (conditional): JS mean=0.191302 min=0.128613 max=0.293481 n=8 |
    branch: js0~0.130533, js1~0.327427, w0_bar~0.694, w1_bar~0.306
k=4-7 (multi-k): JS mean=0.026171 min=0.025292 max=0.027255 n=8

```

```

[JS-REPORT] aaa vs ba (reps 4 x 8), L=5, k_refine=4
  k=1 (emission): JS mean=0.076284 min=0.071615 max=0.082653 n=32
  k=4 (conditional): JS mean=0.158762 min=0.067144 max=0.187838 n=32 |
    branch: js0~0.177043, js1~0.003527, w0_bar~0.894, w1_bar~0.106
  k=4-7 (multi-k): JS mean=0.052067 min=0.048860 max=0.056617 n=32

[JS-REPORT] aaa vs bab (reps 4 x 4), L=5, k_refine=4
  k=1 (emission): JS mean=0.002818 min=0.001707 max=0.004319 n=16
  k=4 (conditional): JS mean=0.169294 min=0.124671 max=0.288750 n=16 |
    branch: js0~0.129190, js1~0.332530, w0_bar~0.804, w1_bar~0.196
  k=4-7 (multi-k): JS mean=0.028672 min=0.027197 max=0.030519 n=16

[JS-REPORT] aaa vs baab (reps 4 x 2), L=5, k_refine=4
  k=1 (emission): JS mean=0.171451 min=0.164475 max=0.179681 n=8
  k=4 (conditional): JS mean=0.175691 min=0.133562 max=0.227148 n=8 |
    branch: js0~0.132685, js1~0.318528, w0_bar~0.766, w1_bar~0.234
  k=4-7 (multi-k): JS mean=0.048184 min=0.045653 max=0.051088 n=8

[JS-REPORT] aaa vs baa (reps 4 x 4), L=5, k_refine=4
  k=1 (emission): JS mean=0.079250 min=0.073607 max=0.085635 n=16
  k=4 (conditional): JS mean=0.027086 min=0.000049 max=0.107322 n=16 |
    branch: js0~0.000202, js1~0.175096, w0_bar~0.844, w1_bar~0.156
  k=4-7 (multi-k): JS mean=0.000327 min=0.000054 max=0.000773 n=16

[JS-REPORT] aaab vs ba (reps 2 x 8), L=5, k_refine=4
  k=1 (emission): JS mean=0.079282 min=0.074148 max=0.085282 n=16
  k=4 (conditional): JS mean=0.178506 min=0.157102 max=0.234918 n=16 |
    branch: js0~0.160217, js1~0.273435, w0_bar~0.838, w1_bar~0.162
  k=4-7 (multi-k): JS mean=0.084605 min=0.083122 max=0.087017 n=16

[JS-REPORT] aaab vs bab (reps 2 x 4), L=5, k_refine=4
  k=1 (emission): JS mean=0.003457 min=0.002138 max=0.004972 n=8
  k=4 (conditional): JS mean=0.000093 min=0.000061 max=0.000175 n=8 |
    branch: js0~0.000082, js1~0.000115, w0_bar~0.710, w1_bar~0.290
  k=4-7 (multi-k): JS mean=0.000111 min=0.000065 max=0.000214 n=8

[JS-REPORT] aaab vs baab (reps 2 x 2), L=5, k_refine=4
  k=1 (emission): JS mean=0.175697 min=0.168114 max=0.183347 n=4
  k=4 (conditional): JS mean=0.000185 min=0.000118 max=0.000259 n=4 |
    branch: js0~0.000110, js1~0.000223, w0_bar~0.734, w1_bar~0.266
  k=4-7 (multi-k): JS mean=0.000099 min=0.000058 max=0.000157 n=4

[JS-REPORT] aaab vs baa (reps 2 x 4), L=5, k_refine=4
  k=1 (emission): JS mean=0.082300 min=0.076172 max=0.088306 n=8
  k=4 (conditional): JS mean=0.112864 min=0.071910 max=0.139373 n=8 |
    branch: js0~0.134129, js1~0.032536, w0_bar~0.789, w1_bar~0.211
  k=4-7 (multi-k): JS mean=0.032423 min=0.030980 max=0.034009 n=8

[JS-REPORT] ba vs bab (reps 8 x 4), L=5, k_refine=4
  k=1 (emission): JS mean=0.050908 min=0.046108 max=0.056485 n=32
  k=4 (conditional): JS mean=0.173970 min=0.160172 max=0.231991 n=32 |
    branch: js0~0.161854, js1~0.278226, w0_bar~0.898, w1_bar~0.102
  k=4-7 (multi-k): JS mean=0.089572 min=0.087649 max=0.092418 n=32

[JS-REPORT] ba vs baab (reps 8 x 2), L=5, k_refine=4

```

```

k=1 (emission): JS mean=0.021430 min=0.018498 max=0.024077 n=16
k=4 (conditional): JS mean=0.176285 min=0.164151 max=0.197192 n=16 |
    branch: js0~0.164080, js1~0.265085, w0_bar~0.874, w1_bar~0.126
k=4-7 (multi-k): JS mean=0.121869 min=0.119446 max=0.125445 n=16

[JS-REPORT] ba vs baa (reps 8 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.000058 min=0.000000 max=0.000237 n=32
k=4 (conditional): JS mean=0.179508 min=0.161482 max=0.194100 n=32 |
    branch: js0~0.183927, js1~0.133431, w0_bar~0.918, w1_bar~0.082
k=4-7 (multi-k): JS mean=0.075352 min=0.070143 max=0.082418 n=32

[JS-REPORT] bab vs baab (reps 4 x 2), L=5, k_refine=4
k=1 (emission): JS mean=0.133820 min=0.126049 max=0.141683 n=8
k=4 (conditional): JS mean=0.000261 min=0.000136 max=0.000417 n=8 |
    branch: js0~0.000140, js1~0.000440, w0_bar~0.781, w1_bar~0.219
k=4-7 (multi-k): JS mean=0.000150 min=0.000105 max=0.000221 n=8

[JS-REPORT] bab vs baa (reps 4 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.053373 min=0.047733 max=0.058992 n=16
k=4 (conditional): JS mean=0.118276 min=0.075954 max=0.138714 n=16 |
    branch: js0~0.132734, js1~0.034618, w0_bar~0.852, w1_bar~0.148
k=4-7 (multi-k): JS mean=0.034919 min=0.032735 max=0.037400 n=16

[JS-REPORT] baab vs baa (reps 2 x 4), L=5, k_refine=4
k=1 (emission): JS mean=0.019866 min=0.017079 max=0.022915 n=8
k=4 (conditional): JS mean=0.121447 min=0.098026 max=0.144141 n=8 |
    branch: js0~0.136221, js1~0.029075, w0_bar~0.861, w1_bar~0.139
k=4-7 (multi-k): JS mean=0.055022 min=0.051733 max=0.058479 n=8

```